Android 13 SDK 基础架构
说明书

版本号: 1.0
发布日期: 2022.11.27

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2022.11.27	AWA0418	初始版本

广州星翼电子科技有限公司zdyzlinux

目　录

1 前言 **1**
1.1 文档简介 1
1.2 目标读者 1
1.3 适用范围 1

2 Android 13 模块架构介绍 **2**

3 多媒体 **4**
3.1 多媒体中间件 libcedarx 4
3.2 中间件中各模块功能 4
3.3 多媒体框架 5
3.4 多媒体代码结构 5

4 显示 **8**
4.1 显示系统架构 8
4.2 显示系统代码结构 8

5 音频 **9**
5.1 音频系统架构 9
5.2 音频代码结构 10

6 Camera **11**
6.1 Camera 系统架构 11
6.2 Camera 代码结构 12
6.2.1 JAVA 应用层 12
6.2.2 Android 本地框架层 12
6.2.3 Android 硬件抽象层接口 13
6.2.4 Android 硬件抽象层 13

7 OTA/Recovery **14**
7.1 OTA/Recovery 代码结构 14
7.1.1 Recovery Aosp 代码结构 14
7.1.2 制作 OTA 升级包目录结构 14

8 网络 WIFI **15**
8.1 网络 WIFI 框架 15
8.2 网络 WIFI 代码结构 16

9 蓝牙 **17**
9.1 蓝牙系统框架图 17
9.2 蓝牙模块代码目录 17

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10 Codec2 **19**
10.1 Codec2 简介 19
10.2 Codec2 框架 19
10.3 Codec2 代码结构 21
10.3.1 Android 中 Codec2 分层 21
10.3.2 Android 中 Codec2 源码结构 21

广州星翼电子科技有限公司zdyzlinux

插　图

图 2-1　Android 架构图 3
图 3-1　多媒体框架图 5
图 4-1　显示系统架构图 8
图 5-1　音频架构图 9
图 6-1　Camera 架构图 11
图 8-1　网络 WIFI 框架图 15
图 9-1　蓝牙系统架构图 17
图 9-2　蓝牙源码结构图 17
图 10-1　Codec2 架构图 20

广州星翼电子科技有限公司zdyzlinux

1 前言

1.1 文档简介

Android 13 系统自下而上分别是 Linux 内核 (Linux Kernel)，系统库 (Libraries)，Android 运行时环境 (Android Runtime), 框架层 (Application Framework) 以及应用层 (Application)。本文主要是介绍基于 Android 13 系统的架构性设计，模块进行划分介绍，旨在快速了解 Android 13 各模块基础结构与代码分布。

1.2 目标读者

SDK 平台负责人、SDK 开发人员、模块开发人员。

1.3 适用范围

Allwinner Android 13 及以上平台。

广州星翼电子科技有限公司zdyzlinux

2 Android 13 模块架构介绍

以模块角度划分,Android 13 可划分为以下部分：

- 多媒体
- 显示
- 音频
- Camera
- OTA/Recovery
- 网络 Wifi
- 蓝牙
- Codec2

Android 架构图如下

广州星翼电子科技有限公司zdyzlinux

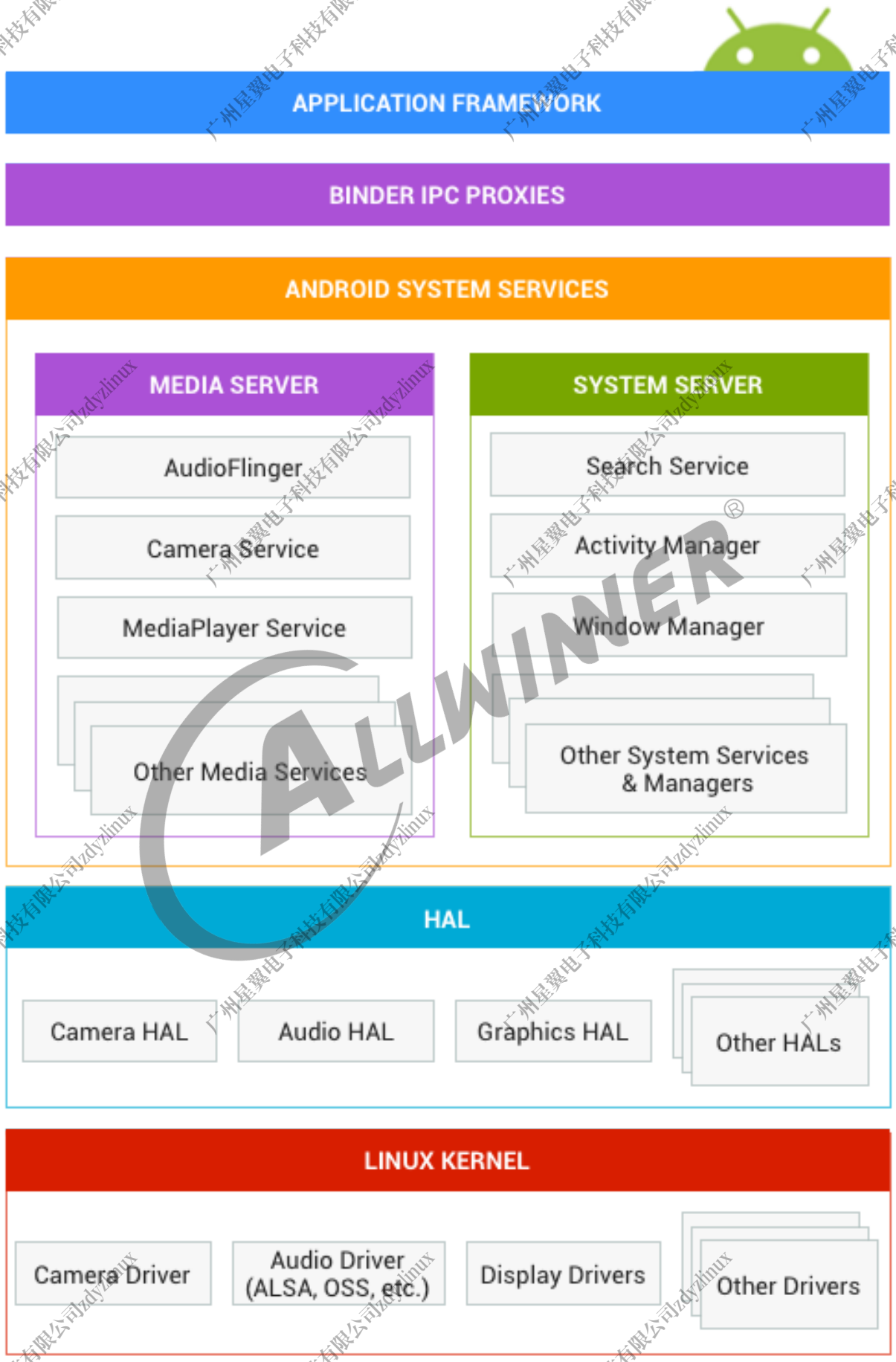

图 2-1: Android 架构图

3 多媒体

3.1 多媒体中间件 libcedarx

多媒体 cedarx 中间件是全志科技设计实现音视频编解码与播放控制的软件模块。主要模块包括接口层，流控模块，解封装模块，回放模块。

- 接口层：awplayer 是多媒体中间件 cedarx 在 android 平台上对接 mediaPlayer 的适配层。
- 流控模块：stream 模块将对本地或网络片源进行加载和处理，目前支持本地片源流以及通过 hls,http,rtsp 等网络协议传输的片源流。
- 解封装模块：parser 模块将对片源码流进行解封装操作，将片源码流分离为音频、视频、字幕流送给音频、视频、字幕解码库进行解码。
- 回放模块：playback 控制解码及音视频字幕的解码和送显。包括音频解码模块 audioDecComponent, 视频解码模块 videoDecComponent, 字幕解码模块 subtitleDecComponent；render 是渲染模块，音频渲染模块 audioRenderComponent, 视频渲染模块 videoRenderComponent, 字幕渲染模块 subtitleRenderComponent 六个模块。

3.2 中间件中各模块功能

- 接口层（android_adapter）：接口层中包括 awplayer, IPTV 的调用接口。接口层中包括 awplayer 播放器的接口文件，接收播放器 apk 发下来各种操作，并派发到 cedarx 的 demuxComponent 和 player 中进行处理。
- 中间件的播放器（xplayer）：xplayer 与接口层 awplayer 对接，响应播放操作的逻辑，控制 cedarx 的 demuxComponent 和 player 进行具体的播放逻辑。
- 流控模块（stream）：stream 模块即 cedarx 的流控模块，负责片源码流数据的加载和处理。stream 是播放的对象也是 parser 解封装的对象。
- 解封装模块（parser）：parser 模块即 cedarx 的接封装模块，parser 模块将对片源码流进行解封装操作，将片源码流分离为音频、视频、字幕流送给音频、视频、字幕解码库进行解码。
- 播控模块模块（playback）：player 是播控模块中的控制函数。实现了音视频字幕模块的加载与初始化，并控制码流的分发和调用音视频字幕解码模块进行解码，然后在音视频字幕解码接收模块做音视频同步处理并输出到屏幕与音响进行播放。主要包括音频解码模块包括音频解码模块 audioDecComponent, 视频解码模块 videoDecComponent, 字幕解码模块 subtitleDecComponent；音频解码渲染模块 audioRenderComponent, 视频解码渲染模块 videoRenderComponent, 字幕解码接渲染模块 subtitleRenderComponent 六个模块。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.3 多媒体框架

我司多媒体框架如下图所示。

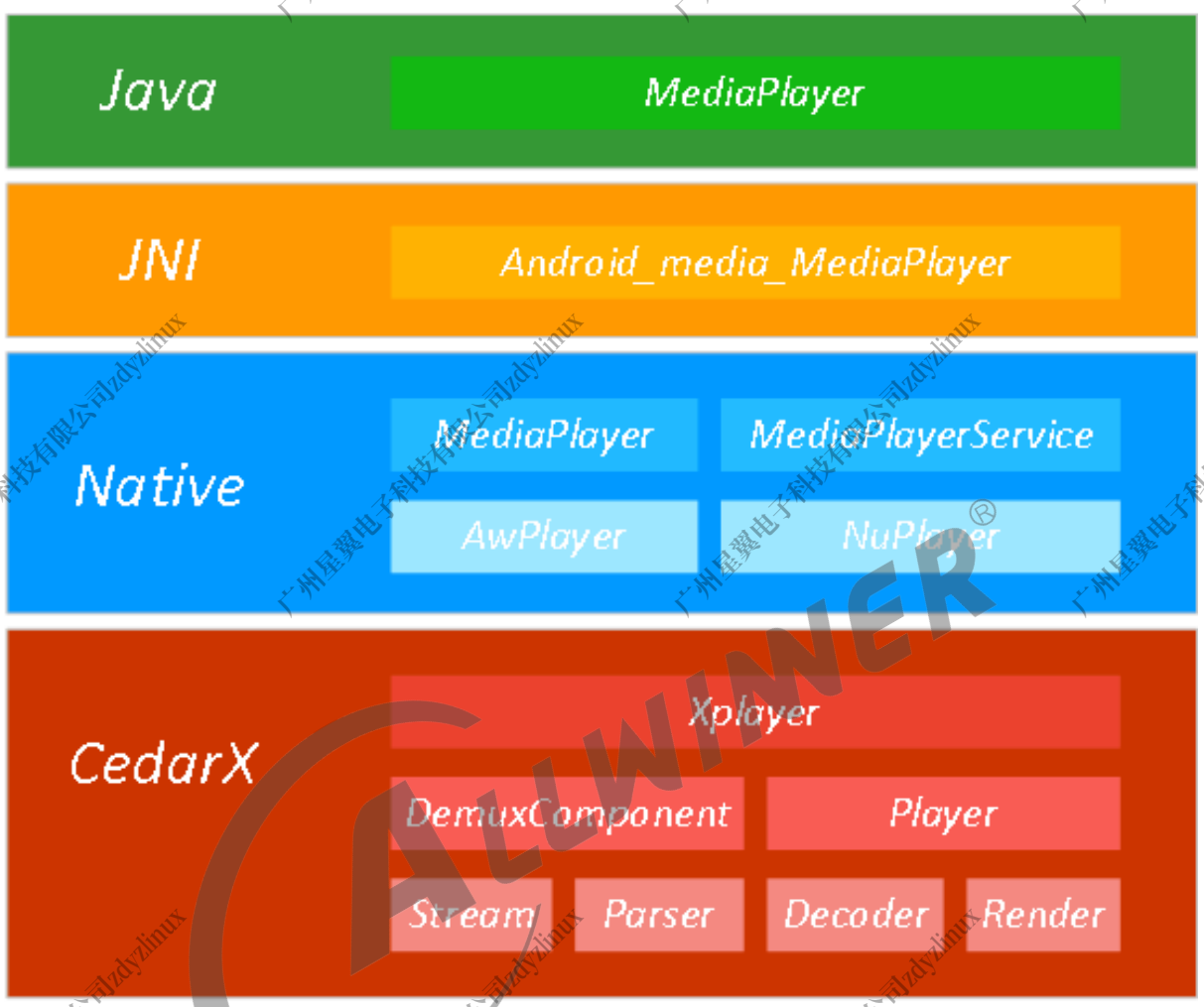

图 3-1: 多媒体框架图

3.4 多媒体代码结构

1. Android 多媒体模块，java 层和 jni 层代码目录。

```
frameworks/base/media
 ├── aidl
 ├── aidl_api
 ├── java
 ├── jni
 ├── lib
 ├── mca
 ├── native
 ├── packages
 └── tests
```

2. Android 多媒体模块，Native 层代码目录。

```
frameworks/av/media
 ├─── audioserver
 ├─── bufferpool
 ├─── codec2
 ├─── codecs
 ├─── common_time
 ├─── extractors
 ├─── img_utils
 ├─── janitors
 ├─── libaaudio
 ├─── libaudioclient
 ├─── libaudiofoundation
 ├─── libaudiohal
 ├─── libaudioprocessing
 ├─── libcedarc
 ├─── libcedarx
 ├─── libcpustats
 ├─── libdatasource
 ├─── libeffects
 ├─── liberror
 ├─── libheadtracking
 ├─── libheif
 ├─── libmedia
 ├─── libmediaformatshaper
 ├─── libmediahelper
 ├─── libmediametrics
 ├─── libmediaplayerservice
 ├─── libmediatranscoding
 ├─── libnbaio
 ├─── libnblog
 ├─── libshmem
 ├─── libstagefright
 ├─── libwatchdog
 ├─── mediaserver
 ├─── mtp
 ├─── ndk
 ├─── tests
 └─── utils
```

3. CedarX 多媒体中间件目录。

```
android/frameworks/av/media/libcedarx
 ├─── android_adapter
 │    ├─── awplayer
 │    ├─── base
 │    ├─── iptv
 │    ├─── metadataretriever
 │    └─── output
 ├─── awrecorder
 ├─── conf
 ├─── config
 ├─── demo
 ├─── document
 ├─── external
 ├─── libcore
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
│   ├──── base
│   ├──── common
│   ├──── muxer
│   ├──── parser
│   ├──── playback
│   └──── stream
├──── xmetadataretriever
└──── xplayer
```

4. CedarC 多媒体编解码库目录。

```
android/frameworks/av/media/libcedarc
├──── base
├──── conf
├──── config
├──── include
├──── library
├──── memory
├──── openmax
│   ├──── include
│   ├──── libstagefrighthw
│   ├──── omxcore
│   ├──── vdec
│   └──── venc
├──── vdecoder
└──── vencoder
```

广州星翼电子科技有限公司zdyzlinux

4 显示

4.1 显示系统架构

显示系统架构图如下所示。

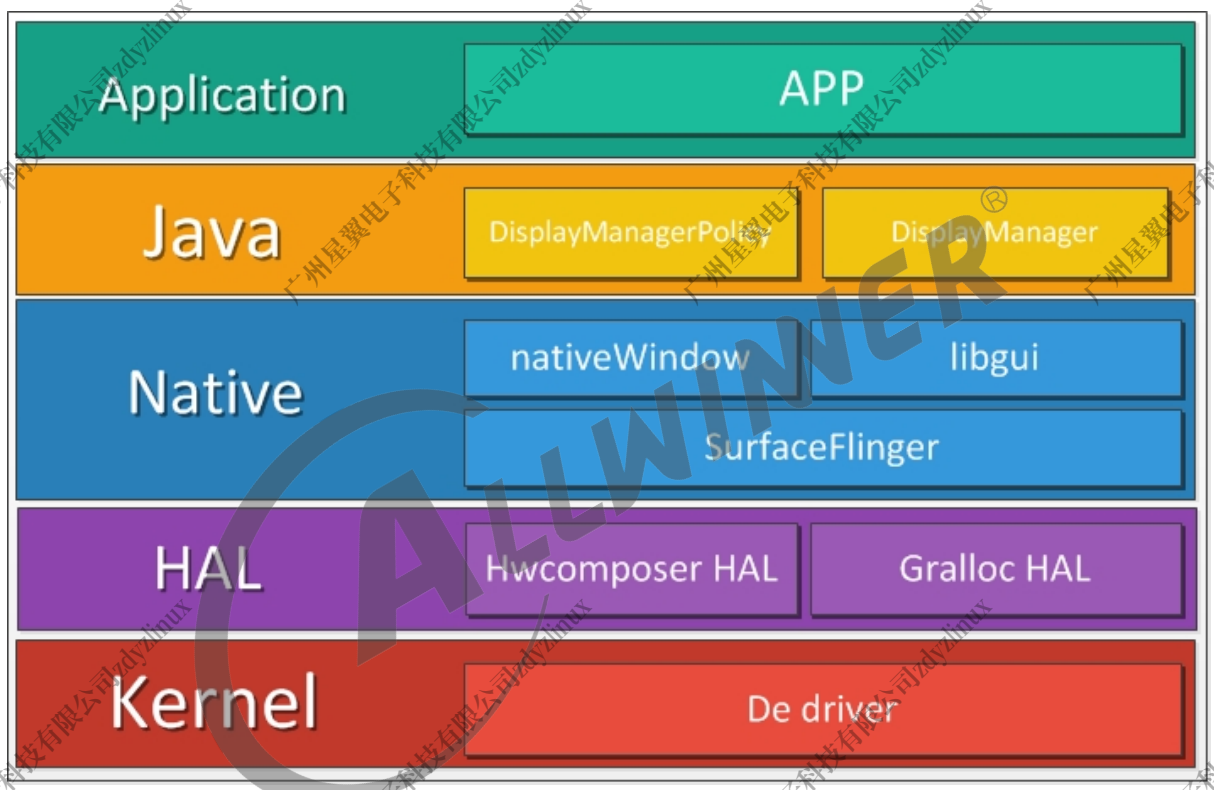

图 4-1: 显示系统架构图

4.2 显示系统代码结构

```
vendor/aw/public/package/displayoutput/
frameworks/native/libs/gui/
frameworks/native/server/surfaceflinger/
hardware/aw/display/
hardware/aw/gpu/
longan/bsp/drivers/video/sunxi/disp2/
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 音频

5.1 音频系统架构

我司音频系统架构图如下所示。

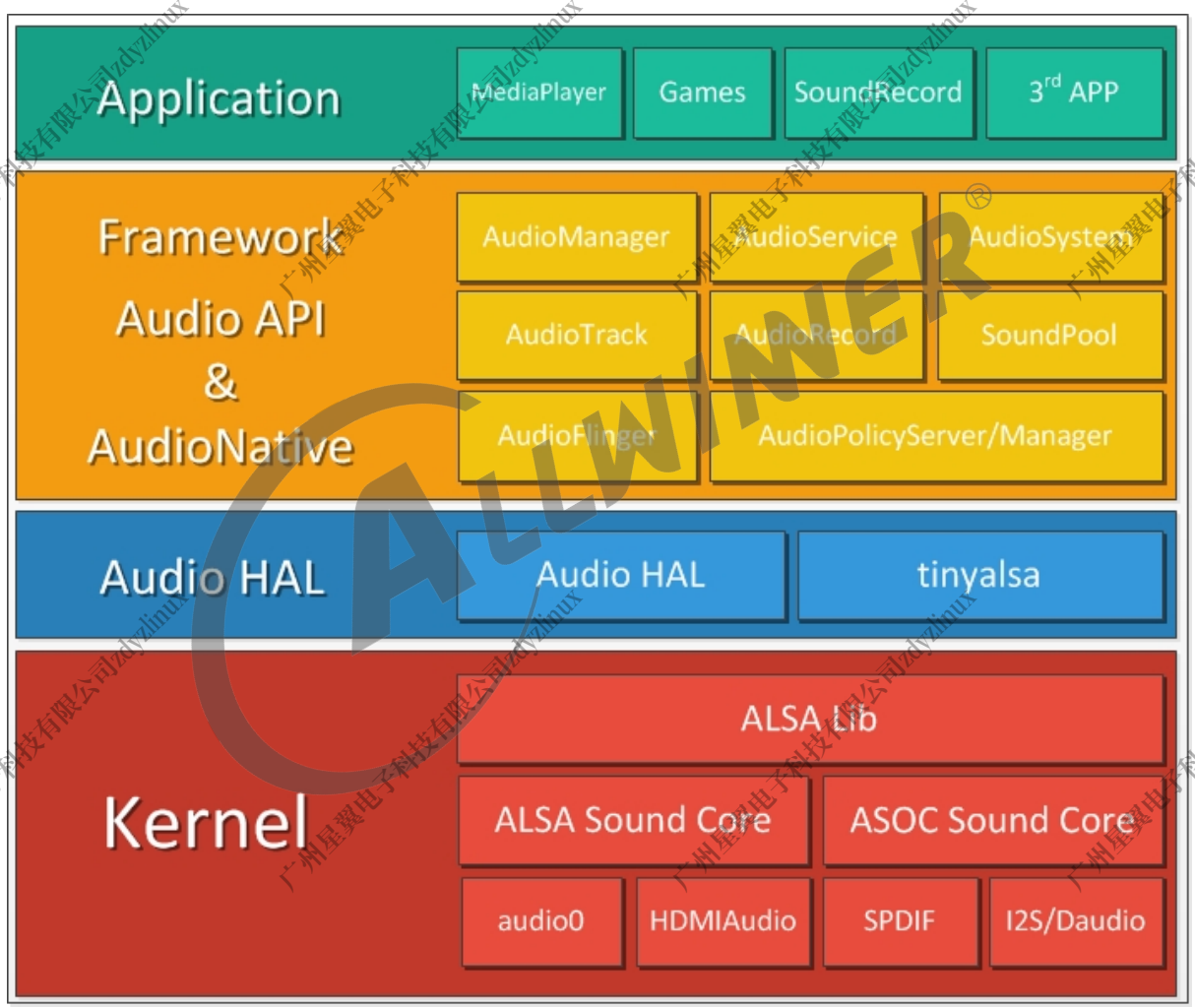

图 5-1: 音频架构图

广州星翼电子科技有限公司zdyzlinux

5.2 音频代码结构

framework:
android/frameworks/base
1.AudioManager.java ---音频管理器，音量调节、音量UI、设置和获取参数等控制流的对外API
2.AudioService.java ---音频系统服务（java层），音量调节、音量UI、音频设备插拔等控制流的具体实现
3.AudioSystem.java ---音频控制的入口，是native层对上服务的接口
android/frameworks/av
1.AudioFlinger.cpp ---音频系统的核心一，承担音频数据流AudioTrack和AudioRecord的混音、重采、输送等责任
2.AudioPolicyService.cpp ---音频系统的核心二，负责音频策略，包含Audio HAL的加载，音频路由的选择等

HAL:
android/hardware/aw/audio/ ---AudioFlinger与音频驱动之间的对接层，匹配android系统与硬件的关键层

广州星翼电子科技有限公司zdyzlinux

6 Camera

6.1 Camera 系统架构

我司 Camera 架构图如下图所示。

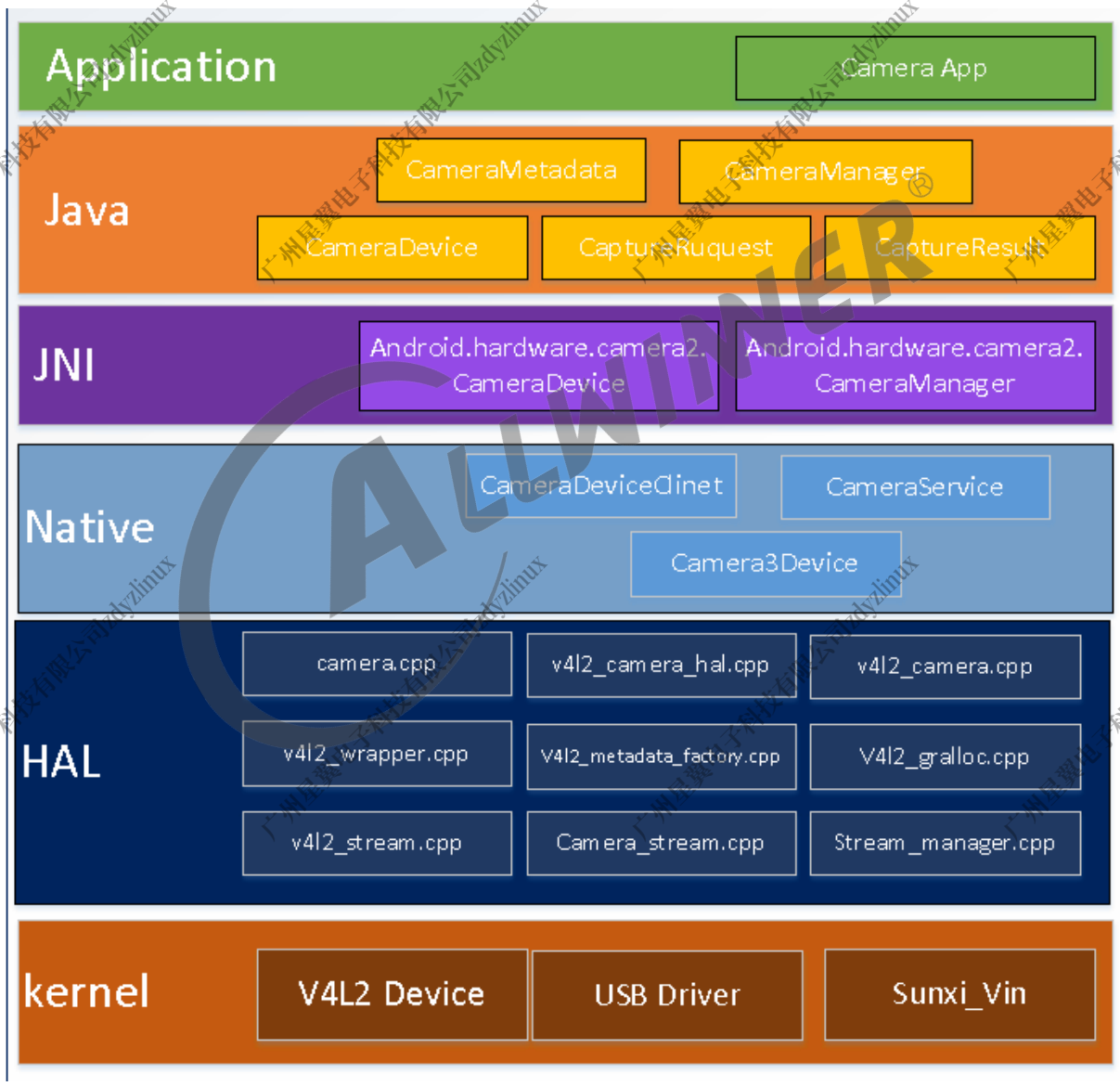

图 6-1: Camera 架构图

6.2 Camera 代码结构

6.2.1 JAVA 应用层

提供如下 Camera 应用源码

AWCamera：

```
android/vendor/aw/public/package/apk/AWCamera
```

AOSP Camera：

```
packages/apps/Camera2
```

它主要调用下面的原生接口进行实现。

```
Camera.java(android/frameworks/base/core/java/android/hardware)
```

6.2.2 Android 本地框架层

Android 本地框架层是 JAVA 层与硬件抽象层的桥梁，它提供了客户端与服务器端通过 IBinder 机制进行进程间通信的模型，极大的方便了上层与下层之间的沟通。它的主要代码文件夹为。

```
frameworks/av/camera/
frameworks/av/include/camera/
frameworks/av/services/camera/libcameraservice/
```

Android Camera 模块的本地框架的主要头文件包括。

```
1. Camera.h
2. CameraMetadata.h
3. CpatureResult.h
4. CaptureRequest.h
5. OutputConfiguration.h
6. ICamera.h
7. ICameraClient.h
8. ICameraService.h
9. CameraHardwareInterface.h
```

主要 cpp 文件包括。

```
1. Camera.cpp
2. CameraService.cpp
3. CameraClient.cpp
4. Camera2Client.cpp
5. CameraMetadata.cpp
6. CaptureResult.cpp
7. Camera3Device.cpp
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
8. Camera3Stream.cpp
```

Native framework 的定义位于 frameworks/av/camera/Camera.cpp 中，它提供了 android.hardware.Camera 类在本地的实现。这些类调用 IPC binder 来获得 camera service。

IPC binder 代理可以现实进程间通信。在 frameworks/av/camera 目录下有三个 camera binder 类的定义。ICameraService 是 cameraservice 的接口，ICamera 是被打开的 camera 设备的接口，ICameraClient 是 camera 设备返回给 application framework 层的接口。

6.2.3 Android 硬件抽象层接口

Android 的硬件抽象层接口在以下路径中，主要为上层提供了可以对硬件抽象层进行操作的接口。

```
frameworks/av/services/camera/libcameraservice/device3
```

6.2.4 Android 硬件抽象层

Camera 的 HAL 层路径为。

```
(hardware/aw/camera/)
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 OTA/Recovery

OTA 即空中下载技术 (over-the-air), 指 Android 系统提供的标准软件升级方式，即通过无线网络下载更新包并无损地升级系统，而无需通过有线方式进行连接。

7.1 OTA/Recovery 代码结构

7.1.1 Recovery Aosp 代码结构

```
├──── bootable/recovery
└──── recovery.cpp   #recovery进程入口
└──── screen_ui.cpp
└──── ui.cpp         #负责Recovery的显示功能
└──── bootloader.cpp   #负责操作bootloader
└──── updater
  └──── install.cpp  #负责定义OTA升级中安装脚本的语句,厂商可在此扩展脚本接口。
└──── applypatch
  └──── applypatch.c #负责在差分升级中通过打patch形式将旧文件升级到新文件
└──── verifier.cpp     #负责提供校验OTA包的方法
└──── roots.cpp        #提供挂载外部设备方法
└──── default_device.cpp#负责控制Recovery界面菜单
```

7.1.2 制作 OTA 升级包目录结构

```
└──── build/tools/releasetools
  └──── ota_from_target_files #利用编译生成的targetfile文件生成OTA完全包或者差分包
  └──── img_from_target_files #利用编译生成的targetfiles生成包括system.img,boot.img,recovery.img等镜像
  └──── sign_target_files_apk #用于对targetfiles中的apk进行签名
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 网络 WIFI

8.1 网络 WIFI 框架

我司网络 WIFI 框架图如下所示。

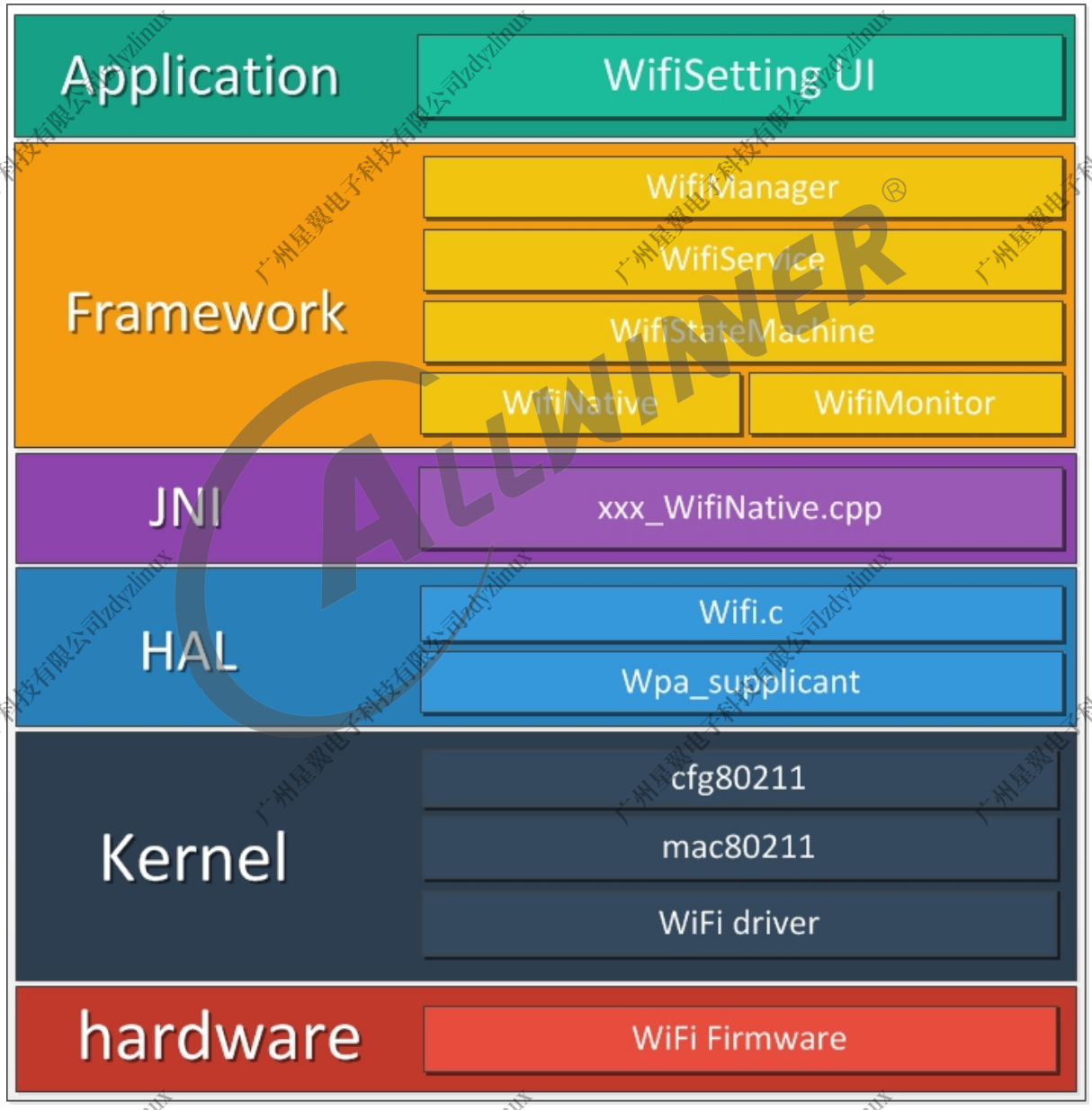

图 8-1: 网络 WIFI 框架图

1. WifiSetting ui 是 WiFi 的界面运用 APP，通过界面的操作，调用到 Framework 层的 API，此 API

由 WifiManager 提供。

2. WifiManger 提供 API 给应用层用，然后接口函数通过异步通道 mAsyncChannel 发送信息给 WifiService 或是通过包含直接调用 WifiService 里面的方法。
3. WifiStateMachine 是 WiFi framework 的逻辑控制中心，管理 WiFi 的各种状态。
4. WifiNative 和 jni 是 frameworks 与 hal 层的沟通桥梁，将 WiFi 的操作命令传递给 wpa_supplicant。
5. WifiMonitor 通过阻塞等到 wpa_supplicant 反馈的命令。
6. wpa_supplicant 是 WiFi 驱动和 Android 层的中转站，wpa_supplicant 通过 nl80211 将命令传递给驱动，同时也负责对协议和加密认证的支持。

8.2 网络 WIFI 代码结构

层次	路径
APP	packages/apps/Settings/src/com/android/settings/wifi
framework	frameworks/base/wifi
jni	frameworks/base/core/jni/android_net_wifi_WifiNative.cpp
hal	hardware/interfaces/wifi
wpa_supplicant	external/wpa_supplicant_8
driver	longan/bsp/drivers/net/wireless
firmware	hardware/aic/wlan/firmware
firmware	hardware/broadcom/wlan/bcmdhd/firmware
firmware	hardware/sprd/wlan/firmware
firmware	hardware/xradio/wlan/kernel-firmware

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9 蓝牙

9.1 蓝牙系统框架图

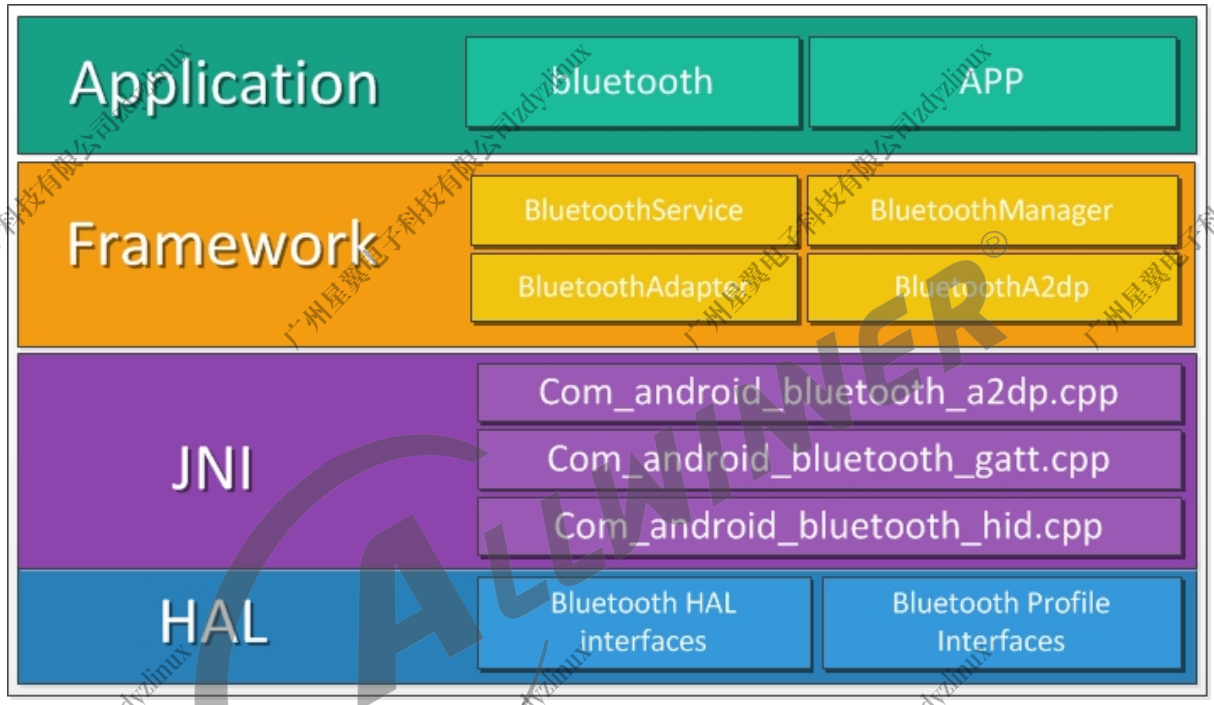

图 9-1: 蓝牙系统架构图

9.2 蓝牙模块代码目录

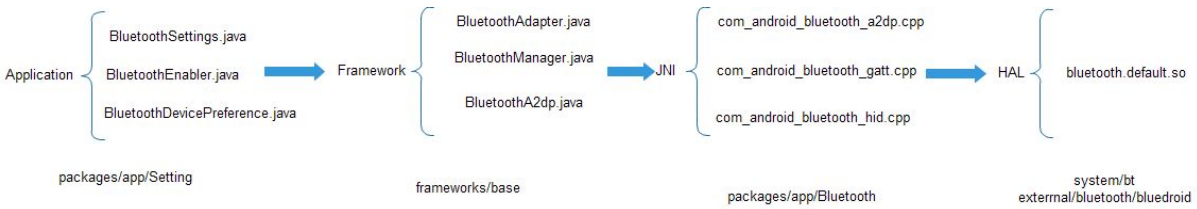

图 9-2: 蓝牙源码结构图

1. 模块涉及的 AOSP 原生代码目录。

```
└──── hardware/interfaces/bluetooth
Broadcom蓝牙模组接口层
└──── packages/apps/Bluetooth
蓝牙进程，实现蓝牙服务和JNI部分，负责衔接frameworks和蓝牙协议栈。
└──── frameworks/base/core/java/android/bluetooth
蓝牙API。
```

2. 模块涉及的 SOC 厂家适配代码目录。

```
├──── hardware/broadcom/libbt                # broadcom 接口库
├──── hardware/broadcom/wlan/bcmdhd/firmware     # broadcom Wi-Fi/BT 固件
├──── hardware/aw/wireless/partner/ampak/firmware # Ampak(broadcom chip) Wi-Fi/BT 固件
├──── hardware/realtek/bluetooth/libbt-vendor    # realtek 接口库
├──── hardware/realtek/bluetooth/firmware        # realtek BT 固件
├──── hardware/xradio/bt/libbt-vendor            # xradio 接口库
└──── hardware/xradio/bt/firmware                # xradio BT 固件
├──── hardware/aic/libbt                     # aic 接口库
└──── hardware/aic/wlan/firmware/            # aic Wi-Fi/BT 固件
```

在支持蓝牙语音通话功能时会涉及到音频模块，涉及到音频模块目录有。

```
android/hardware/aw/audio
```

广州星翼电子科技有限公司zdyzlinux

10 Codec2

10.1 Codec2 简介

在 Android Q 之前，Android 的两套多媒体框架分别为 MediaPlayer 与 MediaCodec，后者只负责解码与渲染工作，解封装工作由 MediaExtractor 代劳，MediaCodec 经由 ACodec 层调用第三方编解码标准接口 OpenMAX IL，实现硬件编解码。芯片厂商只需要支持上 Khronos 制定的 OpenMAX 接口，就可以实现 MediaCodec 的硬件编解码。谷歌在 Android Q 上推出了 Codec2.0，指在于取代 ACodec 与 OpenMAX，它可以看作是一套新的对接 MediaCodec 的中间件，往上对接 MediaCodec Native 层，往下提供新的 API 标准供编解码使用，相当于 ACodec 2.0。

10.2 Codec2 框架

我司 Android Codec2 框架如下图所示。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

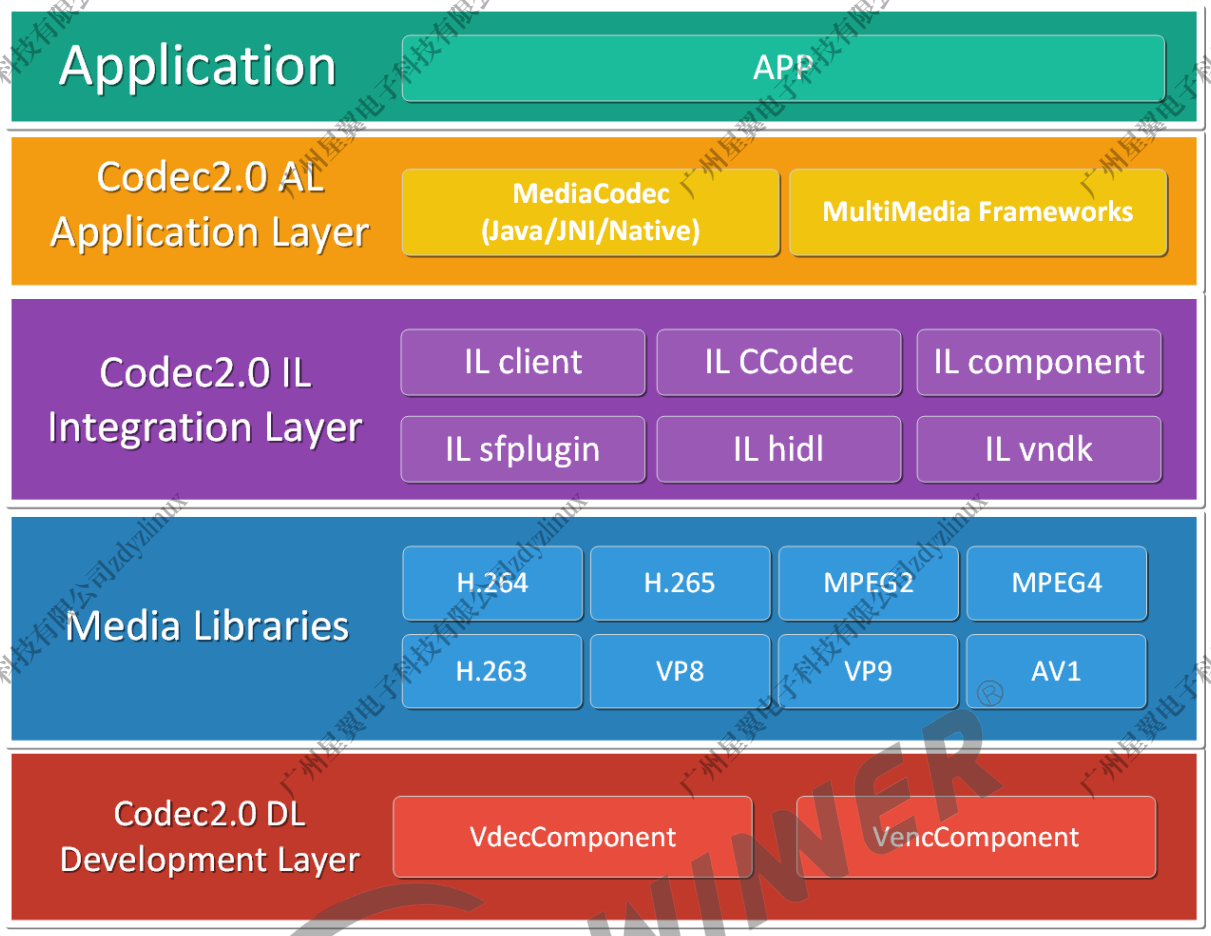

图 10-1: Codec2 架构图

对标 OpenMax 的框架标准，Codec2 也可以分为三层。

1. Codec2.0 AL(Application Layer 应用层) 这一层是多媒体应用和多媒体中间层的标准接口，它使得多媒体应用在多媒体接口上具有可移植性。Codec2 AL 层是 Codec2 规范 API 集得最上层接口，对于上层多媒体应用的开发只需要关注 Codec2 AL 层的接口，因此开发者只需要调用 AL 层的相应接口函数 (MediaCodec) 就可以完成对多媒体的开发。
2. Codec2.0 IL(Integration Layer 应用层) 这一层使得多媒体应用和多媒体框架可以以统一的方式访问多媒体编解码组件和底层的组件，多媒体编解码组件可以是硬件编解码和软件编解码。在架构底层上位多媒体的编解码和数据处理定义了一套统一的编程接口，为用户屏蔽了底层的细节。IL 的主要目的是使用特征集合为编解码器提供一个系统抽象，为解决多个不同媒体系统之间轻便性的问题。
3. Codec2.0 DL(Development Layer 应用层) 这一层包含了视频、音频、图像编解码使用的函数集合，这些函数可以由芯片或硬件厂商对新处理器进行实现和优化，然后编解码供应商使用它来编写更广泛的编解码器功能。它包括音频信号的处理功能，如 FFT 和 filter，图像原始处理，如颜色空间转换、视频原始处理，以实现例如 MPEG-4、H.264、MP3、AAC 和 JPEG 等编解码器的优化。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

10.3 Codec2 代码结构

10.3.1 Android 中 Codec2 分层

MediaCodec 分为 3 部分：Java、JNI 和 Native。

1. Java 是上层 apk 使用的接口。
2. JNI 是 Java 访问 Native 的 JNI 方法。
3. Native 是 CCodec 的 wrapper。

10.3.2 Android 中 Codec2 源码结构

Codec2.0 的代码目录位于/frameworks/av/media/codec2。目录结构如下。

```
codec2
|--components  #具体编解码组件与组件接口层
|   |--base/SimpleC2Component.cpp
|   |--base/SimpleC2Interface.cpp
|   |--avc/C2SoftAvcDec.cpp
|--core      #存在核心的头文件，譬如Buffer定义、Component定义、Config定义、Param定义
|--docs      #暂时存放doxygen配置文件与脚本
|--faultinjection
|--fuzzer
|--hidl      #与hidl调用相关的实现
  |--client/client.cpp
  |--1.0/utils/Component.cpp
  |--1.0/utils/ComponentInterface.cpp
  |--1.0/utils/ComponentStore.cpp
  |--1.0/utils/Configurable.cpp
  |--1.0/utils/include/codec2/hidl/1.0/Component.h
  |--1.0/utils/include/codec2/hidl/1.0/Configurable.h
  |--1.0/utils/include/codec2/hidl/1.0/ComponentInterface.h
  |--1.1/*
  |--1.2/*
|--sfplugin   #顶层接口与实现层
|   |--CCodec.cpp
|   |--CCodec.h
|   |--CBufferChannel.cpp
|   |--CBufferChannel.h
|--tests
|--vndk      #基础的util实现
|   |--C2Store.cpp
```

我司的 Codec2.0 实现目录位于/hardware/aw/libcodec2。目录结构如下。

```
|-- aidl
|-- components
|   |-- decoders #解码组件
|   |   |-- avc
|   |   |   |-- AvcDecComponent.cpp
|   |   |-- gav1
```

```
| | | |-- C2SoftGav1Dec.cpp
| | |-- h263
| | | |-- H263DecComponent.cpp
| | |-- hevc
| | | |-- HevcDecComponent.cpp
| | |-- mjpeg
| | | |-- MjpegDecComponent.cpp
| | |-- mpeg1
| | | |-- Mpeg1DecComponent.cpp
| | |-- mpeg2
| | | |-- Mpeg2DecComponent.cpp
| | |-- mpeg4
| | | |-- Mpeg4DecComponent.cpp
| | |-- vp8
| | | |-- Vp8DecComponent.cpp
| | |-- vp9hwal
| | | |-- Vp9DecComponent.cpp
| |-- encoders #编码组件
| | |-- avc
| | | |-- AvcEncComponent.cpp
| | |-- hevc
| | | |-- HevcEncComponent.cpp
| | |-- jpeg
| |     |-- MjpegEncComponent.cpp
| |-- HwC2Component.cpp
| |-- HwC2Interface.cpp
| |-- VdecComponent.cpp # 解码组件具体实现
| |-- VencComponent.cpp # 编码组件具体实现
|-- config
|-- plugins
  |-- widevine_plugins
|-- services # HAL层服务实现
|-- vndk
  |-- C2Store.cpp
```

著作权声明

版权所有 ©2023 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

、、全志科技、（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。